ĐẠI HỌC
BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY
OF SCIENCE AND TECHNOLOGY

TRAVELING SALESMAN PROBLEM WITH TIME WINDOWS

Group 3

Member 1: Le Khanh Bao Minh - 202516596
Member 2: Pham Dinh Nhat Minh - 202516602
Member 3: Nguyen Huy Tu - 202516684
Member 4: Nguyen Phu Cuong - 202516647

ONE LOVE. ONE FUTURE.

Table of contents

Introduction to TSPTW

A delivery worker starts from a warehouse (point 0) and must deliver goods to N customers located at points 1 through N.

- Each customer i has a delivery time window from e_i to l_i
- Each delivery takes d_i units of time
- The travel time between any two points i and j is $c_{i,j}$

Objective: Find a delivery route that minimizes travel time while ensuring that all deliveries are made within their respective time windows.

Note 1: We model the route as a closed loop $[0, s[1], \ldots, s[N], 0]$
Note 2: We let $e_0 = 0, l_0 = \max_{i \in \{1,\ldots,N\}}(l_i + d_i + c_{i,0}), d_0 = 0$

Branch and Bound

Branch and Bound is an exact algorithm that systematically enumerates candidate solutions via recursive backtracking. It guarantees finding the optimal route by evaluating all feasible permutations while actively pruning branches that cannot possibly yield a better solution than the currently known best sequence. To improve the performance of Branch and Bound, we have two following optimizations.

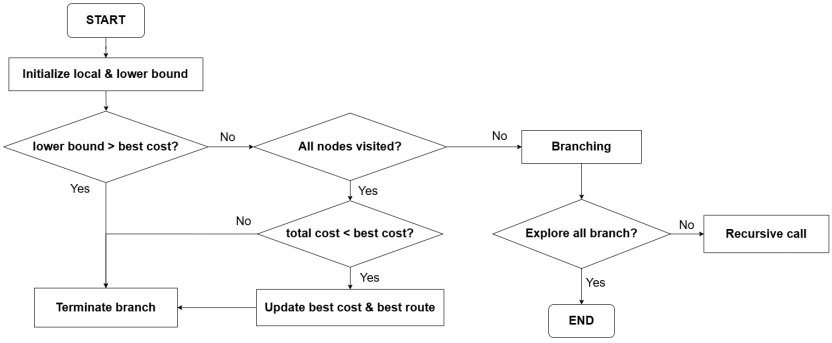
1. From the current node, the algorithm generates branches to unvisited candidate nodes, only consider those whose projected arrival time is sooner than the deadline.

$$e_i + c_{i,j} + d_i \leq l_j$$

2. The algorithm computes a lower bound (LB) by adding the current total cost with an optimistic projection of the remaining journey. If the $LB \geq$ Best Cost, the branch is pruned.

$$LB = \text{current cost} + (N - level) \cdot c_{min}$$

Branch and Bound



In this model, let:

- N: number of customers
- $c_{i,j}$: the travel time from point i to point j
- $e_i,\ l_i,\ d_i$: the earliest time, latest time, and service duration at point $i \in \{1, 2, \ldots, N\}$

Decision variables:

- $x_{i,j} \in \{0, 1\}$: 1 if the route goes directly from i to j, 0 otherwise
- $t_i \in \mathbb{N}$: arrival time at point i

- **Graph sparsification**

$$A = \{(i,j) \in \{0,1,\ldots,N\}^2 \mid i \neq j \text{ and } (j = 0 \vee e_i + c_{i,j} + d_i \leq l_j)\}$$

We only consider arcs from i to j ($i \neq j$) if it satisfies one out of two following conditions:

1. $j = 0$: It is always possible to return to the depot (point 0).
2. $e_i + c_{i,j} + d_i \leq l_j$: The earliest possible departure from i ($e_i + d_i$) + travel time ($c_{i,j}$) does not violate the deadline of j (l_j).

- **Hamiltonian circuit constraint**

$$Circuit(\{x_{i,j} \mid (i,j) \in A\})$$

The arcs must form a Hamiltonian cycle.

- **Time constraints**

$$e_i \leq t_i \leq l_i \qquad \forall\, i = 1, \ldots, N$$

The arrival time at point i must belong to the window $[e_i, l_i]$.

$$t_j \geq t_i + d_i + c_{i,j} \qquad \forall\, (i,j) \in A \text{ if } x_{i,j} = 1 \text{ and } j \neq 0$$

The arrival time at point j must be greater than or equal to the earliest possible departure from i $(e_i + d_i)$ + travel time $(c_{i,j})$.

Objective function

$$\min \sum_{(i,j) \in A} c_{i,j} \cdot x_{i,j}$$

In this model, we let

- N: number of customers
- $c_{i,j}$: the travel time from point i to point j
- $e_i,\ l_i,\ d_i$: the earliest time, latest time, and service duration at point $i \in \{1, 2, \ldots, N\}$

Decision variables

- $x_{i,j} \in \{0, 1\}$: 1 if the route goes from i to j, 0 otherwise
- $t_i \in \mathbb{R}_{\geq 0}$: arrival time at customer i
- $u_i \in \{1, \ldots, N\}$: sequencing variable for subtour elimination

- **Graph sparsification**

$$A = \{(i,j) \in \{0,1,\ldots,N\}^2 \mid i \neq j \text{ and } (j = 0 \vee e_i + c_{i,j} + d_i \leq l_j)\}$$

We only consider arcs from i to j ($i \neq j$) if it satisfies one out of two following conditions:

1. $j = 0$: It is always possible to return to the depot (point 0).
2. $e_i + c_{i,j} + d_i \leq l_j$: The earliest possible departure from i ($e_i + d_i$) + travel time ($c_{i,j}$) does not violate the deadline of j (l_j).

- **Flow conservation**

Each point i has exactly one outgoing arc, and each point j has exactly one incoming arc.

$$\sum_{j|(i,j)\in A} x_{i,j} = 1 \qquad \forall\, i \in \{0,1,\ldots,N\} \qquad \sum_{i|(i,j)\in A} x_{i,j} = 1 \qquad \forall\, j \in \{0,1,\ldots,N\}$$

- **Time constraints**

$$e_i \leq t_i \leq l_i \qquad \forall\, i = 1, \ldots, N$$

The arrival time at point i must belong to the window $[e_i, l_i]$.

$$t_j \geq t_i + d_i + c_{i,j} - M_{i,j}(1 - x_{i,j}) \qquad \forall\, (i,j) \in A \text{ and } j \neq 0$$

where

$$M_{i,j} = \max(0, l_i - e_j + d_i + c_{i,j})$$

- **Subtour elimination (MTZ constraint)**

$$u_i - u_j + N \cdot x_{i,j} \leq N - 1 \qquad \forall\, (i,j) \in A \text{ with } i \neq 0 \text{ and } j \neq 0$$

Objective function

$$\min \sum_{(i,j) \in A} c_{i,j} \cdot x_{i,j}$$

Heuristic algorithms

Because exact solvers (like MIP and CP) struggle with exponentially growing search spaces, this heuristic relies on a rapid **Greedy Phase** to find an initial feasible route, followed by an **Local Search Phase** to iteratively refine the objective cost. This approach balance computational efficiency with high solution quality.

Greedy Algorithm

- At any current node i, a candidate node j is considered feasible only if the vehicle can arrive before its deadline. Let $\text{arrival}_j = t + d_i + c_{i,j}$, the condition is: $\text{arrival}_j \leq l_j$.
- For each feasible candidate j, the algorithm looks at these details:
  - **Start Time:** $\text{start}_j = \max(\text{arrival}_j,\ e_j)$
  - **Finish Time:** $\text{finish}_j = \text{start}_j + d_j$
  - **Waiting Time:** $\text{wait}_j = \text{start}_j - \text{arrival}_j$
  - **Slack:** $\text{slack}_j = l_j - \text{arrival}_j$
- The algorithm builds the sequence by greedily picking the candidate j that minimizes the combined score H. This score looks at travel time, waiting time, and how close the deadline is:

$$H = c_{i,j} + \text{wait}_j + 0.35 \cdot \text{slack}_j$$

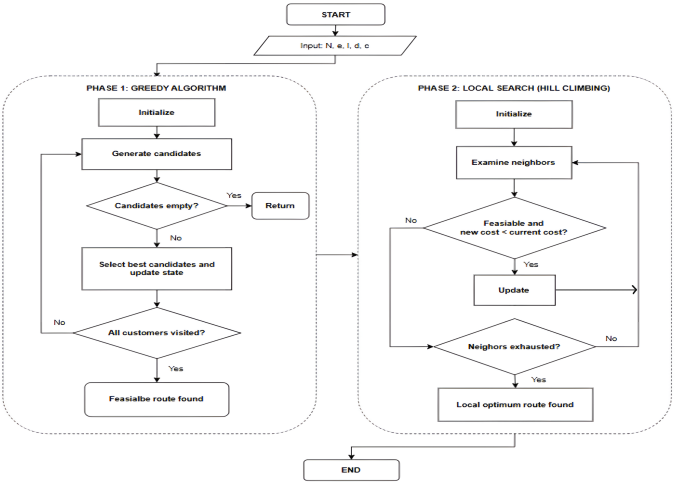
$$\min_{j \in \text{candidates}} H$$

Local Search Algorithm

To refine the initial suboptimal route of the Greedy construction, a **Hill Climbing** algorithm is applied.

- For an initial route, the algorithm generates neighboring solutions by **Node Swapping**: iterating over all possible pairs of nodes in the sequence and swapping their positions.
- The algorithm uses a **First Improvement** rule. It immediately accepts the first valid swap that results in a temporally feasible route with a strictly lower cost, halting the current neighborhood search.
- The search then restarts, using this newly improved route as the baseline to explore its neighbors.
- The algorithm terminates when it reaches a **Local Minimum** where no further feasible improvements are found.

Combined Heuristic Framework



While the heuristic approach provides computationally fast and feasible initial routes, local search mechanisms are inherently prone to getting trapped in local optima, particularly due to the highly rugged search space created by time-window constraints. To achieve near-optimal solutions for larger instances, more sophisticated exploration strategies are required.

This section introduces two advanced metaheuristic frameworks employed to overcome these limitations: **Ant Colony Optimization (ACO)** and **Adaptive Large Neighborhood Search (ALNS)**.

Ant Colony Optimization (ACO)

Ant Colony Optimization is a population-based metaheuristic inspired by the foraging behavior of real ants. In the context of the TSPTW, ants iteratively construct delivery routes. They make probabilistic routing decisions based on two factors: **pheromone trails** (representing the historical success of previous ants on a path) and **heuristic information** (representing the immediate, greedy desirability of visiting a node). Over multiple iterations, the pheromone trails are updated to reinforce shorter, feasible routes.

The ACO model is controlled by the following set of parameters:

- $\tau_{i,j}$: amount of pheromone on (i, j)
- $\eta_{i,j}$: heuristic information of (i, j)
- Q: pheromone deposit factor
- α: importance of pheromone
- β: importance of heuristic information
- ρ: pheromone evaporation rate

- Heuristic Information - η
  The heuristic information $\eta_{i,j}$ guides ants toward nodes that make logical sense based on time constraints and travel cost. It relies on the combined heuristic score $H_{i,j}$ defined previously in the Greedy phase. To convert this cost into a desirability metric (where higher is better), it is inverted:

$$\eta_{i,j} = \frac{1}{H_{i,j} + \epsilon}$$

  where ϵ is a small constant used to prevent division by zero.

- State Transition Rule
  An ant at node i chooses the next node $j \in A_i$ based on the probability $P_{i,j}$:

$$P_{i,j} = \frac{[\tau_{i,j}]^{\alpha} \cdot [\eta_{i,j}]^{\beta}}{\sum_{k \in A_i} [\tau_{i,k}]^{\alpha} \cdot [\eta_{i,k}]^{\beta}}$$

- Pheromone Update Rules
  In each iteration, after all ants have traversed, the amount of pheromone on each edge is updated. To prevent the algorithm from converging instantly on a suboptimal route, a **Min-Max Ant System (MMAS)** approach is used alongside an **elitist** update strategy.
  1. **Evaporation:** At the end of each iteration, pheromones on all edges evaporate to slowly forget older, potentially poor routes:

$$\tau_{i,j} \leftarrow (1 - \rho) \cdot \tau_{i,j}$$

2. **Elitist Deposit:** Only the ant that found the best route in the current iteration is allowed to deposit pheromone. The update rule is defined as:

$$\tau_{i,j} \leftarrow \tau_{i,j} + \Delta\tau_{i,j}$$

where the deposited pheromone $\Delta\tau_{i,j}$ is calculated piecewise:

$$\Delta\tau_{i,j} = \begin{cases} \frac{Q}{L_{\text{best}}} & \text{if } (i,j) \in \text{Best Route} \\ 0 & \text{otherwise} \end{cases}$$

and L_{best} represents the total travel cost of that iteration's best route.

3. **Min-Max Bounds:** To maintain exploration and prevent the probabilities from locking into a single path, all pheromone values are strictly bounded within a dynamic range $[\tau_{\min}, \tau_{\max}]$. These bounds are updated based on the global best route cost ($L_{\text{global\_best}}$):

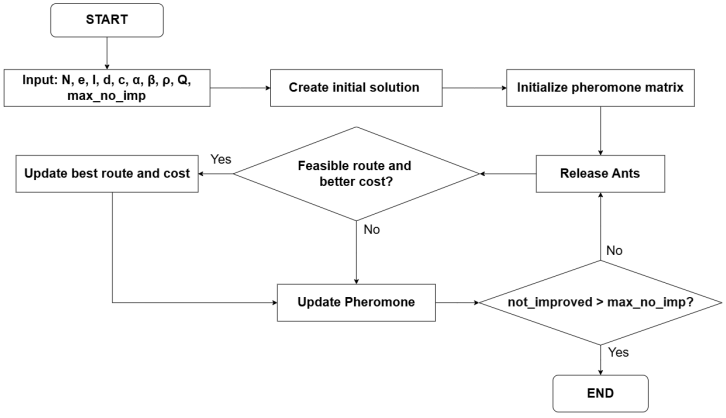
$$\tau_{\max} = \frac{Q}{\rho \cdot L_{\text{global\_best}}}$$

$$\tau_{\min} = \frac{1}{N} \cdot \tau_{\max}$$

After the elitist deposit, the pheromone trails are clamped to respect these limits:

$$\tau_{i,j} \leftarrow \max(\tau_{\min}, \min(\tau_{\max}, \tau_{i,j}))$$

Ant Colony Optimization (ACO)



Adaptive Large Neighborhood Search (ALNS)

Adaptive Large Neighborhood Search is a metaheuristic that iteratively improves a solution by partially **destroying** and subsequently **repairing** it. Compared to Local Search, ALNS explores significantly larger search spaces, allowing the algorithm to escape local optima more effectively. In the context of TSPTW, the algorithm relies on a competitive pool of destroy and repair operators, dynamically learning which operators perform best during the search process.

The ALNS model is controlled by the following parameters:

- q: number of nodes removed per iteration
- T: the temperature controlling SA acceptance rate
- α: the cooling rate for T

Adaptive Large Neighborhood Search (ALNS)

- **Destroy Operators**: Random Removal, Segment Removal, Worst Removal
- **Repair Operators**: Random Insertion, Greedy Insertion, Regret-2 Insertion
- **Objective Funtion**:

$$f(s) = \sum c_{u,v} + M \cdot \sum \max(0, t_v - l_v)$$

where t_v is the actual arrival time at node v, and M is a massive penalty scalar used to heavily discourage late arrivals. This is used to allow the algorithm to temporary explore infeasible regions to find better optimum.

- **SA Acceptance Criterion**
  A newly generated route s' with cost $f(s')$ that is strictly worse than the current route s is accepted with probability:

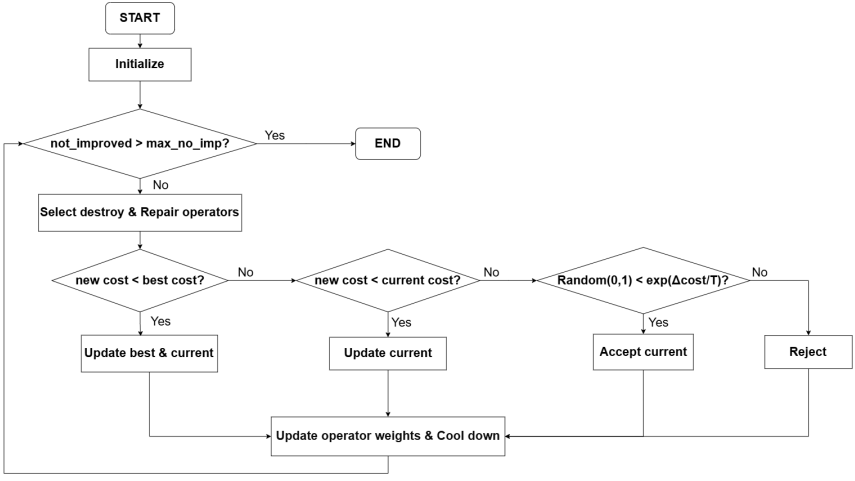
$$P = e^{-\frac{f(s') - f(s)}{T}}$$

- **Adaptive Weight Adjustment**
  After an iteration, the applied operators are scored based on the outcome of the newly generated route:
  - $\sigma = 33$: The route is a new global best.
  - $\sigma = 13$: The route is non-improving but accepted via Simulated Annealing criterion.
  - $\sigma = 9$: The route is better than the current route.
  - $\sigma = 0$: The route is rejected.

  The weights are then updated using a decay factor $\lambda \in [0, 1]$ to ensures the algorithm retains a memory of past performance while reacting to the current phase of the search:

$$w_i \leftarrow \lambda \cdot w_i + (1 - \lambda) \cdot \sigma$$

Adaptive Large Neighborhood Search (ALNS)



Hustack test cases

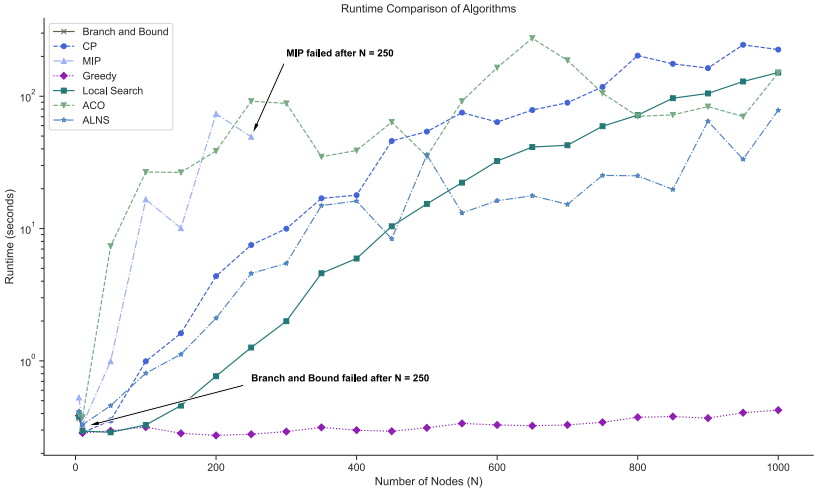
N	Branch & Bound		CP		MIP		Greedy		Local Search		ACO		ALNS	
	Cost	Time(s)	Cost	Time(s)	Cost	Time(s)	Cost	Time(s)	Cost	Time(s)	Cost	Time(s)	Cost	Time(s)
5	380	0.37	380	0.41	380	0.47	380	0.38	380	0.29	380	0.31	380	0.46
10	620	0.28	620	0.28	620	0.30	620	0.26	620	0.28	620	0.36	620	0.37
100	TLE	TLE	56310	0.32	56310	0.38	56310	0.26	56310	0.27	56310	5.15	56310	0.96
200	TLE	TLE	98410	0.49	98410	0.69	98410	0.27	98410	0.27	98410	10.73	98410	1.69
300	TLE	TLE	160090	0.87	160090	1.23	160090	0.29	160090	0.30	160090	16.61	160090	3.50
500	TLE	TLE	259360	2.03	259360	3.26	259360	0.32	259360	0.45	259360	31.59	259360	10.66
600	TLE	TLE	326720	2.84	326720	4.79	326720	0.31	326720	0.55	326720	41.06	326720	15.03
700	TLE	TLE	357980	3.98	357980	6.44	357980	0.32	357980	0.63	357980	56.30	357980	21.17
900	TLE	TLE	454330	7.29	454330	11.06	454330	0.36	454330	0.98	454330	74.12	454330	36.81
1000	TLE	TLE	510850	9.68	510850	14.54	510850	0.40	510850	1.22	510850	87.33	510850	43.98

- Branch and Bound failed to compute anything beyond $N = 10$.
- All remaining algorithms managed to achieve optimality in a reasonable time, with Greedy being the fastest and ACO being the slowest.

Code-generated test cases

N	Branch & Bound		CP		MIP		Greedy		Local Search		ACO		ALNS	
	Cost	Time(s)	Cost	Time(s)	Cost	Time(s)	Cost	Time(s)	Cost	Time(s)	Cost	Time(s)	Cost	Time(s)
5	2371	0.37	2371	0.39	2371	0.53	2377	0.39	2371	0.38	2371	0.40	2371	0.42
10	4688	0.32	4688	0.28	4688	0.33	5211	0.28	4688	0.28	4688	0.37	4688	0.33
50	TLE	TLE	23954	0.36	23954	0.99	28205	0.30	24842	0.28	24935	7.36	23954	0.46
100	TLE	TLE	46420	0.99	46420	16.60	53601	0.32	46991	0.33	49393	26.74	46430	0.81
150	TLE	TLE	74608	1.62	74608	10.07	85747	0.28	76277	0.46	82911	26.57	74619	1.12
200	TLE	TLE	92052	4.37	92052	73.56	104698	0.27	92796	0.77	102388	38.58	92308	2.10
250	TLE	TLE	108783	7.51	108784	49.42	124212	0.28	109857	1.26	121002	91.72	108783	4.58
300	TLE	TLE	136141	9.98	TLE	TLE	155882	0.28	137870	1.99	153174	88.26	136220	5.46
350	TLE	TLE	159677	16.93	TLE	TLE	186386	0.31	162434	4.59	185605	34.90	159803	14.93
400	TLE	TLE	186736	17.87	TLE	TLE	213093	0.30	189030	5.94	212152	38.90	187107	16.14
450	TLE	TLE	204739	45.91	TLE	TLE	234134	0.28	209082	10.40	232369	63.82	234134	8.35
500	TLE	TLE	238215	54.04	TLE	TLE	272086	0.31	241260	15.36	271121	35.15	245630	36.04
550	TLE	TLE	237693	75.23	TLE	TLE	281512	0.34	242040	22.25	280723	91.69	275301	13.13
600	TLE	TLE	266592	63.78	TLE	TLE	308278	0.33	270277	32.36	306616	164.54	299872	16.23
650	TLE	TLE	294853	78.68	TLE	TLE	338656	0.32	298798	41.29	336451	274.58	329576	17.71
700	TLE	TLE	328487	89.39	TLE	TLE	380265	0.33	331877	42.62	379000	186.87	380265	15.24
750	TLE	TLE	345919	117.59	TLE	TLE	401122	0.34	351847	59.35	401095	105.35	401122	25.24
800	TLE	TLE	364389	202.78	TLE	TLE	414980	0.37	369860	71.91	414973	70.56	409022	25.04
850	TLE	TLE	381481	175.77	TLE	TLE	441387	0.38	386017	96.64	441078	72.29	434408	19.77
900	TLE	TLE	410156	163.16	TLE	TLE	472303	0.37	413617	104.99	471519	83.56	443597	64.80
950	TLE	TLE	438158	244.80	TLE	TLE	505810	0.41	444472	129.28	505810	70.26	505810	33.39
1000	TLE	TLE	450133	225.18	TLE	TLE	517615	0.42	455099	151.09	517292	149.80	488484	78.38

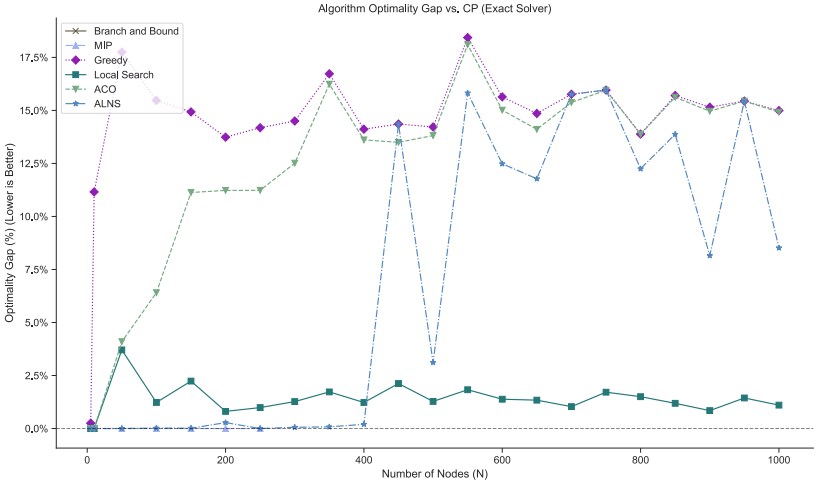
Results and Analysis



Time Analysis

- Greedy remains virtually flat against the bottom of the axis. Even at $N = 1000$, it computes a route in 0.42 seconds, proving it is the only viable option for strictly real-time applications.
- CP is the slowest, scaling up to 225 seconds. Local Search and ACO follow a similar trend, both taking around 150 seconds to complete the 1000-node network.
- ALNS demonstrates phenomenal time scalability. At $N = 1000$, it finishes in just 78 seconds—nearly twice as fast as Local Search and ACO, and three times faster than CP.

Results and Analysis



Cost Analysis

- At small $N \leq 400$, ALNS displays a 0% optimality gap while running faster than CP. However, it fluctuates significantly with increasing N, landing it in the middle.
- ACO fails to provide value at large scale. With $N = 1000$, it takes 150 seconds only to return a cost of 517,292, which is virtually indistinguishable from the instant Greedy route and about 15% higher than the optimal route.
- Local Search's results is incredibly close to the optimal results, staying under 2% deviation from CP for the majority of the graph.

Conclusion

In test cases with tight time windows:

- In these tight time constraints test case, finding a feasible route alone is difficult. Therefore, the only feasible route is usually the optimal route.
- Greedy is the best algorithm here due to it lightning–fast ability to find a feasible route.
- If you want assurance on optimality, consider exact solvers like MIP and CP, as the tight constraints can massively reduce the search space, enabling them to find the optimal route in a reasonable amount of time.

Conclusion

In test cases with loose time windows:

- If you need a mathematically perfect answer, CP is remarkably reliable up to 1,000 nodes, though it takes almost four minutes 225.18 seconds) to finish the largest dataset.
- If you want a near-perfect answer faster, Local Search is your best bet; it delivers an incredibly accurate route (costing 455,099 compared to the perfect 450,133) and saves over a minute of processing time (151.09 seconds).
- ALNS represents a balance between computation time and quality, but it performance varies drastically for large N.
- Greedy is useful if you absolutely need an answer instantly (less than 0.5 seconds for all test cases), but you must be willing to accept a highly inefficient route.

THANK YOU !